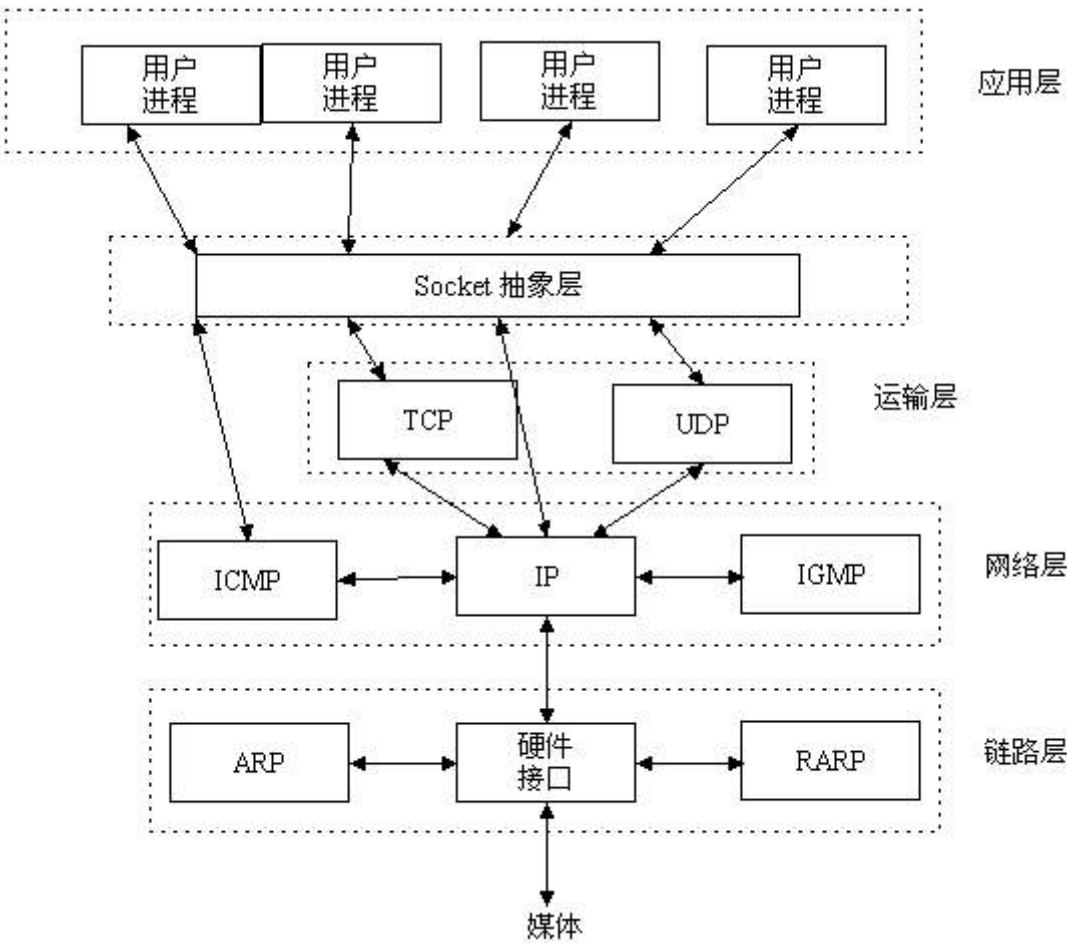


socket api编程

socket api

1、socket编程

TCP/IP协议：IP地址+TCP或UDP端口号，唯一标示网络通讯中的一个进程。称为socket.

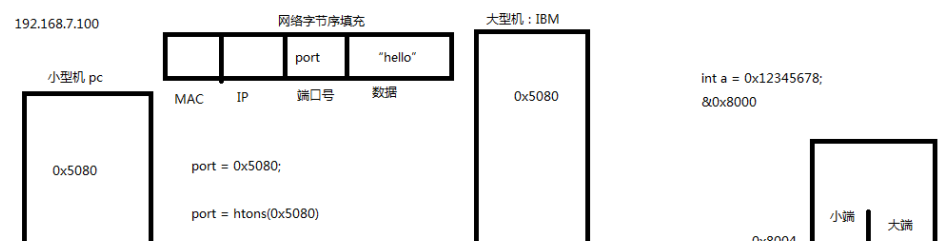


1.1、网络字节序

网络数据流采用大端字节序传输，如果客户端是采用小端字节序，那么就需要把小端字节序转换为大端字节序传输。

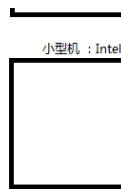
大端字节：低地址高字节，高地址低字节。

小端字节：低地址低字节，高地址高字节。





ntohl
ntohs



小型机：Intel

0x8000	0x78	0x12
0x8001	0x34	0x56
0x8002	0x56	0x34
0x8003	0x12	0x78

传输数据不需要转换。

1.2、网络字节序和主机字节序的转换函数

```
1 #include <arpa/inet.h>
2
3
4 uint32_t htonl(uint32_t hostlong); //小端转换成大端 长整型
5
6
7 uint16_t htons(uint16_t hostshort); //短整型
8
9
10 uint32_t ntohl(uint32_t netlong); //大端转换成小端 长整型
11
12
13 uint16_t ntohs(uint16_t netshort);
14
15
16 h表示host，n表示network，l表示32位长整数，s表示16位短整数。
17 htonl 表示 “host to network long”.
```

长整型函数通常用来转换IP地址，短整型函数用来转换端口号。

如果主机是小端字节序，这些函数将参数做相应的大小端转换然后返回，如果主机是大端字节序，这些函数不做转换，将参数原封不动地返回。

1.3、IP地址转换函数

```
1 #include <sys/socket.h>
```

```

2 #include <netinet/in.h>
3 #include <arpa/inet.h>
4
5
6 int inet_aton(const char *cp, struct in_addr *inp);
7 in_addr_t inet_addr(const char *cp);
8 char *inet_ntoa(struct in_addr in);
9
10
11 只能处理IPv4的ip地址
12 不可重入函数
13 注意参数是struct in_addr

```

更新过的函数接口：

```

1 #include <arpa/inet.h>
2
3
4 int inet_pton(int af, const char *src, void *dst);
5 //这个函数转换字符串到网络地址，
6 //第一个参数af是地址簇，
7 //第二个参数*src是来源地址，
8 //第三个参数* dst接收转换后的数据。
9
10 const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
11
12
13 支持IPv4和IPv6
14 可重入函数

```

1.4、sockaddr数据结构

```

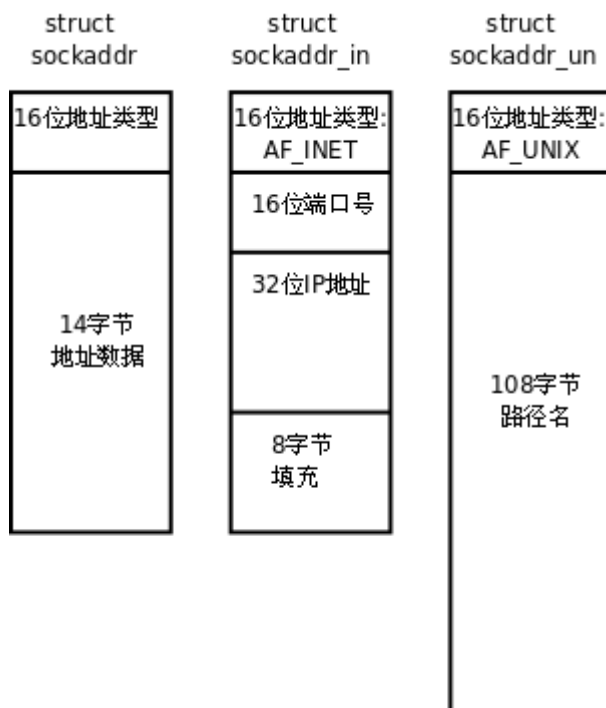
1 struct sockaddr          早期的结构体
2
3
4 struct sockaddr_in       ipv4使用的结构体（用于封装目标IP和端口号）

```

```

5
6
7 struct sockaddr_un      本地套接字

```



1、早期结构体（`struct sockaddr *` 类型，已经变成了泛型（`void *`）类型。）

```

1 struct sockaddr
2 {
3     sa_family_t    sa_family; //表示地址族类型, AF_xxx
4     char          sa_data[14]; //14个字节协议地址
5 };

```

`struct sockaddr *` 是一个通用指针类型结构体，`addr`参数实际上可以接受多种协议的 `sockaddr`结构体，而他们的长度各不相同，所以需要第三个参数`addrlen`指定结构体的长度。

不管ipv4或ipv6，都需要转换成`struct sockaddr *`类型。

2、ipv4结构体

```

1 struct sockaddr_in
2 {
3     __kernel_sa_family_t  sin_family; //表示地址族类型
4     __be16                sin_port; //端口号

```

```

5     struct in_addr      sin_addr; //IP地址
6     unsigned char      __pad[__SOCK_SIZE__ - sizeof(short int) - sizeof(unsigned
7
8
9 };

```

3、地址结构体

```

1 struct in_addr
2 {
3     __be32    s_addr;
4 };

```

地址类型：IPv4\IPv6\Unix Domain Socket的地址类型AF_INET\AF_INET6\AF_UNIX

4、IPv6结构体

```

1 struct sockaddr_in6 {
2     unsigned short int    sin6_family; /* AF_INET6 */
3     __be16                sin6_port; /* Transport layer port # */
4     __be32                sin6_flowinfo; /* IPv6 flow information */
5     struct in6_addr        sin6_addr; /* IPv6 address */
6     __u32                  sin6_scope_id; /* scope id (new in RFC2553) */
7 };

```

5、IPv6地址

```

1 struct in6_addr {
2     union {
3         __u8 u6_addr8[16];
4         __be16 u6_addr16[8];
5         __be32 u6_addr32[4];
6     } in6_u;
7     #define s6_addr      in6_u.u6_addr8
8     #define s6_addr16    in6_u.u6_addr16
9     #define s6_addr32    in6_u.u6_addr32

```

```
10 };
```

6、本地套接字结构体

```
1 #define UNIX_PATH_MAX 108
2 struct sockaddr_un {
3     __kernel_sa_family_t    sun_family; /* AF_UNIX */
4     char                    sun_path[UNIX_PATH_MAX]; /* pathname */
5 };
```

2、网络套接字函数

2.1、socket函数(打开socket)

```
1 #include <sys/types.h> /* See NOTES */
2 #include <sys/socket.h>
3
4
5 int socket(int domain, int type, int protocol);
6
7
8 domain:
9 AF_INET 这是大多数用来产生socket的协议，使用TCP或UDP来传输，用IPv4的地址
10 AF_INET6 与上面类似，不过是用IPv6的地址
11 AF_UNIX 本地协议，使用在Unix和Linux系统上，一般都是当客户端和服务端在同一台及其上的时候使用
12
13
14 type:
15 SOCK_STREAM 这个协议是按照顺序的、可靠的、数据完整的基于字节流的连接。这是一个使用最多的socket类型
16 SOCK_DGRAM 这个协议是无连接的、固定长度的传输调用。该协议是不可靠的，使用UDP来进行它的连接。
17 SOCK_SEQPACKET 这个协议是双线路的、可靠的连接，发送固定长度的数据包进行传输。必须把这个包完整发送
18 SOCK_RAW 这个socket类型提供单一的网络访问，这个socket类型使用ICMP公共协议。（ping、tracert等）
19 SOCK_RDM 这个类型是很少使用的，在大部分的操作系统上没有实现，它是提供给数据链路层使用，不保证顺序
20
21
22 protocol:
23 0 默认协议
```

```
24
25
26 返回值：成功返回一个新的文件描述符，失败返回-1，设置errno
```

socket打开一个网络通讯端口，就像open一样的返回一个文件描述符，应用程序可以像读写文件一样用read\write在网络上收发数据。

2.2、bind函数（绑定socket）

```
1 #include <sys/types.h> /* See NOTES */
2 #include <sys/socket.h>
3
4
5 int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
6
7
8 sockfd:      socket文件描述符
9 addr         构造出IP地址和端口号（struct sockaddr * 类型相当于 void *）
10 addrlen:     sizeof(addr)长度
11
12
13 返回值：成功返回0，失败返回-1，设置errno
```

构造sock结构体：

```
1 struct sockaddr_in  servaddr;
2
3
4 bzero(&servaddr, sizeof(servaddr)); //清零 //memset
5
6
7 servaddr.sin_family = AF_INET; //选择ipv4
8 //INADDR_ANY 表示本地的任意IP地址
9
10 servaddr.sin_addr.s_addr = htonl(INADDR_ANY); //表示本地的任意IP网络地址
11
```

```
12
13 servaddr.sin_port = htons(8000); //端口，在大端序机不需要转换
```

服务器程序所监听的网络地址和端口号通常是固定不变的，客户端程序得知服务器程序的地址和端口号后就可以向服务器发起连接，因此服务器需要调用**bind**绑定一个固定的网络地址和端口号。

2.3、listen函数（监听能链接的个数）

```
1 #include <sys/types.h> /* See NOTES */
2 #include <sys/socket.h>
3
4
5 int listen(int sockfd, int backlog);
6
7
8 sockfd:      socket文件描述符
9 backlog:     排队建立3次握手队列和刚刚建立3次握手队列的链接数和（默认128）
10
11
12 返回值：成功返回0，失败返回-1.
```

查看系统默认backlog `cat /proc/sys/net/ipv4/tcp_max_syn_backlog`

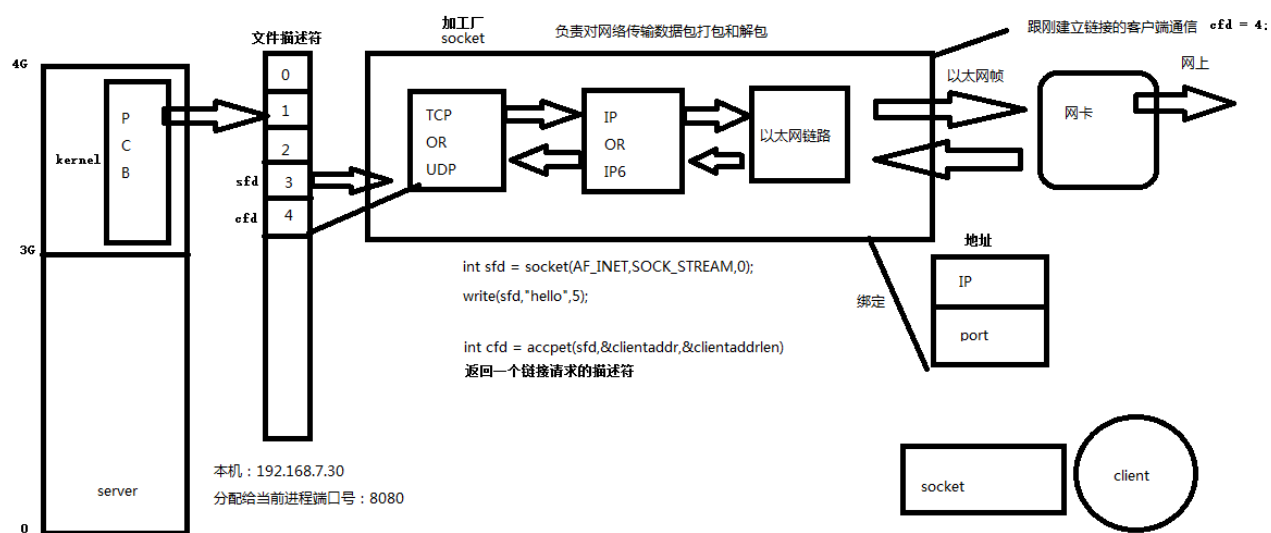
2.4、accept函数（阻塞，被动等待链接）

```
1 #include <sys/types.h> /* See NOTES */
2 #include <sys/socket.h>
3
4
5 int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
6
7
8 sockfd:      socket文件描述符
9 addr:        传出参数，返回链接客户端地址信息，含IP地址和端口号
10 addrlen:     传入传出参数（值-结果），传入sizeof(addr)大小，函数返回时返回真正接收到地址
11
12
```

13 返回值：成功返回一个新的socket文件描述符，用于和客户端通信，失败返回-1，设置errno。

2.5、connect函数(主动链接)

```
1 #include <sys/types.h> /* See NOTES */
2 #include <sys/socket.h>
3
4
5 int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
6
7
8 sockfd:      socket文件描述符
9 addr:        传入参数, 指定服务器端地址信息, 含IP地址和端口号
10 addrlen:     传入参数, 传入sizeof(addr)大小
11 返回值:      成功返回0, 失败返回-1, 设置errno。
```

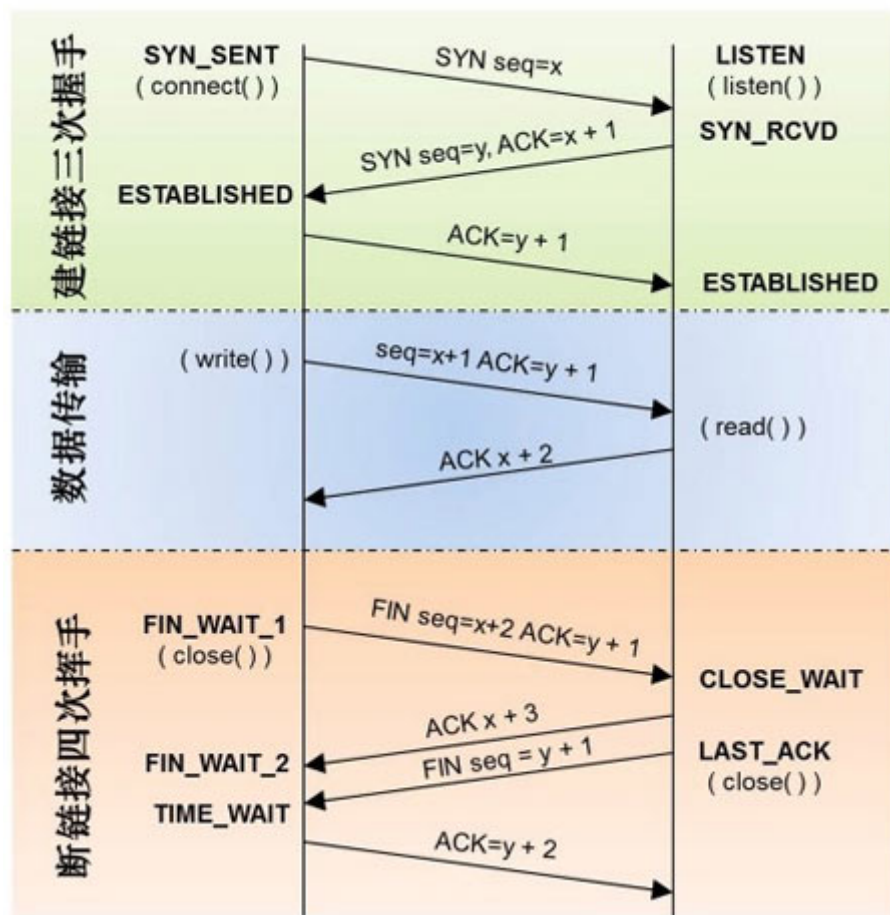


3、TCP通信

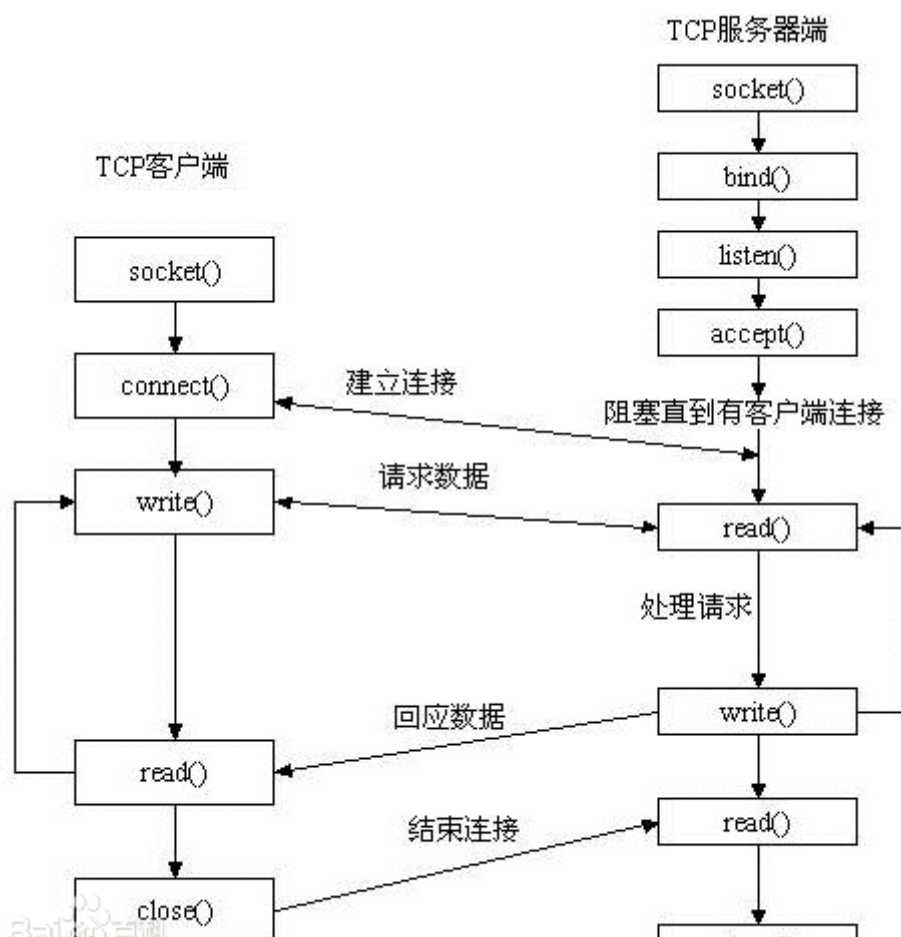
1、建立链接

Client

Server



2、流程图



1、客户端

```
1 #include <stdio.h>
2 #include <strings.h>
3 #include <sys/types.h>
4 #include <sys/socket.h>
5 #include <arpa/inet.h>
6 #include <unistd.h>
7 #include <string.h>
8
9
10 #define SERV_PORT 8080
11 #define SERV_IP "192.168.1.201"
12
13
14 int main(void)
15 {
16     int cfd,len;
17     char buf[4096] = "helloworld";
18
19
20     struct sockaddr_in caddr;
21     cfd = socket(AF_INET,SOCK_STREAM,0); //打开套接字
22
23
24     bzero(&caddr,sizeof(caddr)); //清零处理
25     caddr.sin_family = AF_INET; //设置IPv4
26
27
28     inet_pton(AF_INET,SERV_IP,&caddr.sin_addr.s_addr); //把字符串ip转为大端
29     caddr.sin_port = htons(SERV_PORT); //端口转为大端
30
31
32     connect(cfd,(struct sockaddr *)&caddr,sizeof(caddr)); //链接
33
34
35     write(cfd,buf,strlen(buf)); //写
36     len = read(cfd,buf,sizeof(buf)); //读
37     write(STDOUT_FILENO,buf,len); //打印到屏幕
38     return 0;
```

2、服务器

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <strings.h>
4 #include <sys/types.h>
5 #include <sys/socket.h>
6 #include <arpa/inet.h>
7
8
9 #define SERV_PORT 8080
10
11
12 int main(void)
13 {
14     int sfd,cfd,clen,len,i;
15     struct sockaddr_in saddr,caddr;
16     char ipstr[128],buf[4096];
17
18
19     sfd = socket(AF_INET,SOCK_STREAM,0);//打开套接字
20
21
22     bzero(&saddr,sizeof(saddr));//清零处理
23     saddr.sin_family = AF_INET;//设置IPV4
24     saddr.sin_addr.s_addr = htonl(INADDR_ANY);//任意本机ip地址，转为大端
25     saddr.sin_port = htons(SERV_PORT);//端口号转为大端
26
27
28     bind(sfd,(struct sockaddr *)&saddr,sizeof(saddr));//绑定
29     listen(sfd,20);//监听数量设置
30
31
32     clen = sizeof(caddr);//clen作为传入传出参数
33     cfd = accept(sfd,(struct sockaddr *)&caddr,&clen);//阻塞，等待客户端连接
34
35
```

```
36     printf("client ip = %s\tpport = %d\n",inet_ntop(AF_INET,(void *)&caddr.sin_addr.s_
37     len = read(cfd,buf,sizeof(buf));
38     //小写转大写
39     for(i = 0;i < len;i++)
40         buf[i] = toupper(buf[i]);
41
42
43     write(cfd,buf,len);
44
45
46     close(cfd);
47     close(sfd);
48     return 0;
49 }
```